

CS 133 Lab 2 Report: MPI-Based Parallel GEMM

Name: Alexander Neary

UID: 805935345

Date: April 2026

1. Data and Computation Partitioning

1.1 Data Partitioning

The implementation uses a 1D row-wise decomposition for Matrix A and complete replication for Matrix B.

- **Matrix A:** Processor 0 partitions the rows of A into different and equal chunks based on the number of available processors ($\text{num_rows} = \text{kl} / \text{size}$).
- **Matrix B:** Since every processor needs to access all columns of B to compute the rows of C (using the rows of A they have), the entirety of Matrix B gets sent to every processor.
- **Matrix C:** Similar to Matrix A, each processor only needs to store its results in the num_rows that it's assigned to. The results are combined after the computation of each group of rows per processor is complete.

1.2 Computation Partitioning

The computation is parallelized by assigning each MPI rank a specific subset of rows from Matrix A to calculate the same product rows in Matrix C. Each process independently executes a blocked $i \rightarrow j \rightarrow k$ loop structure to maximize cache reuse and spatial locality.

1.3 Communication Strategy

- **Distribution:** `MPI_Scatter()` is used to send the row chunks of A out to every processor. Similarly, `MPI_Bcast()` is used to send the entire Matrix B to each processor for processing.
 - **Collection:** `MPI_Gather()` is utilized once all computation across processors 0-3 is completed to recombine the rows of C into a full result Matrix C back in processor 0.
-

2. Analysis of Communication APIs

2.1 Theoretical Analysis

- **Blocking (`MPI_Send/Recv`):** Simplest to implement, but can lead to idle time waiting for handshakes to complete and increasing overhead or in the worst case full on deadlocks.

- **Non-Blocking (MPI_Isend/Irecv):** Slightly more complex, allows for communication and computation to overlap so supposedly a new block of B could be fetched while computing the current one.
- **Collective (Scatter/Bcast/Gather):** These were the ideal MPI commands for matrix multiplication as it generalizes the message passing to $p0 \rightarrow \text{all}$ and $\text{all} \rightarrow p0$, which works perfectly for the row-wise decomposition of A, complete matrix-wise replication of B, and gathering of C.

2.2 Experimental Choice

In testing, the choice was to either loop through each process/rank and manually use MPI_Send or MPI_Isend to deliver chunks of Matrix A or the entirety of Matrix B OR to utilize one of the collective commands and have process 0 send everything out at once.

Doing the latter reduced the bottleneck by the root process, resulting in higher performance.

3. Performance Evaluation

3.1 Performance Improvements

- **MPI Infrastructure and 1D Blocking:** Using MPI_Scatter(), MPI_Bcast(), and MPI_Gather() to create a distributed-memory parallel model. ~48 GFlops
- **Better Blocking and I→K→J Loop Order:** Matched the omp.cpp's $i k j$ order to improve memory access performance (rowwise instead of columnwise). ~60 GFlops
- **Updated Back to I→J→K Order and Added Uneven Blocking:** Order allowed for more cache locality with pointers/registers and adjusted block sizes to be unequal, prioritizing rows of B (1024). ~89 GFlops
- **Added Row-Wise Loop Unrolling:** Unroll each row by a factor of 2 to access and perform calculations on more values while they are in cache. ~125 GFlops
- **Double Loop Unrolling Factor and Increase K Block Size:** Unroll each row by a factor of 4 instead of 2 and increase BK_SIZE to 8 from 4. ~135 GFlops

3.2 Different Problem Sizes

The following results were obtained on the **m5.2xlarge** AWS instance.

Problem Size	GFlops	Time (s)	Performance Range
1024 ³	68.7963	0.03121	C
2048 ³	88.4427	0.1942	B
4096 ³	137.0760	1.0026	A

Performance Range achieved: **A**

4. Scalability

Testing was done by varying the number of processors:

- **np=1:** ~41 GFLOps
- **np=2:** ~74 GFLOps
- **np=4:** ~137 GFLOps
- **np=8:** ~128 GFLOps
- **np=16:** ~87 GFLOps
- **np=32:** ~60 GFLOps

Discussion of Non-Linearity: The program scales almost linearly up from 1 processor to 4 processors, but beyond that the performance drops off at a much higher rate. This is likely due to the cost of broadcasting Matrix B across many more cores than with the lower np values.

5. MPI vs OpenMP

5.1 Programming Effort

MPI required a bit more effort, due to the necessity of manual memory allocation (using `aligned_alloc`) and the direct management of which data was being distributed to where (using `Scatter/Bcast/Gather`). Coordinating the collection of results and understanding how/where to use MPI was slightly challenging compared to OpenMP.

5.2 Performance Difference

- **OpenMP (Lab 1):** ~72 GFLOps
- **MPI (Lab 2):** ~137 GFLOps

Analysis: MPI achieved a slightly higher peak performance than OpenMP because there is no shared address space. Each process contains its own private address space and therefore there is minimal risk of processors fighting over the same cache line which is something OpenMP needs to be aware of. OpenMP is probably better for smaller problem sizes as the overhead of message passing becomes detrimental at that point.